

Article

Towards Robust Routing: Enabling Long-Range Perception with the Power of Graph Transformers and Deep Reinforcement Learning in Software-Defined Networks

Xinyuan Li , Junze Li, Jingli Zhou  and Jun Liu *

School of Artificial Intelligence, Beijing University of Posts and Telecommunications, Beijing 100876, China; xinyuanli@bupt.edu.cn (X.L.); lijunze@bupt.edu.cn (J.L.); wayzer@bupt.edu.cn (J.Z.)

* Correspondence: liujun@bupt.edu.cn

Abstract: Deep Reinforcement Learning (DRL) has demonstrated promising capabilities for routing optimization in Software-Defined Networks (SDNs). However, existing DRL-based routing algorithms are struggling to extract graph-structured information and constrained to a fixed topology, suffering from the lack of robustness. In this paper, we strengthen the advantages of Graph Neural Networks (GNNs) for DRL-based routing optimization and propose a novel algorithm named Graph Transformer Star Routing (GTSR) to enhance robustness against topology changes. GTSR utilizes the multi-agent architecture to enable each node to make routing decisions independently, and introduces a Graph Transformer to equip agents with the capabilities of handling topology changes. Furthermore, we carefully design a global message-passing mechanism with a virtual star node and a path-based readout method, enhancing the long-range perception of traffic and the detection of potential congestion for routing decision-making. Moreover, we construct a multi-agent cooperation mechanism to facilitate the learning of universal perceptual strategies and reduce the amount of computation. Extensive experiments on multiple real-world network topologies demonstrate that GTSR is capable of adapting to unseen topology changes without retraining and decreases end-to-end latency by at least 47% and packet loss rate by at least 10% compared to all baselines, highlighting strong robustness.



Academic Editor: Antoni Morell

Received: 16 December 2024

Revised: 17 January 2025

Accepted: 21 January 2025

Published: 24 January 2025

Citation: Li, X.; Li, J.; Zhou, J.; Liu, J.

Towards Robust Routing: Enabling Long-Range Perception with the Power of Graph Transformers and Deep Reinforcement Learning in Software-Defined Networks.

Electronics **2025**, *14*, 476.

<https://doi.org/10.3390/electronics14030476>

Copyright: © 2025 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license

(<https://creativecommons.org/licenses/by/4.0/>).

Keywords: routing optimization; deep reinforcement learning; graph neural network; transformer; software-defined network; robustness

1. Introduction

As a fundamental component of communication networks, routing determines how to transmit data from source to destination, which directly impacts the Quality of Service (QoS). Traditional routing schemes, such as Open Shortest Path First (OSPF) [1] and Equal-Cost Multi-Path (ECMP) [2], typically select fixed shortest paths for transmission and fail to dynamically adjust routing strategies for complex traffic demands. To address this issue, the Software-Defined Network (SDN) [3] has been proposed to enable programmable and flexible configurations, encouraging researchers to design intelligent routing algorithms. Particularly, Deep Reinforcement Learning (DRL) [4] becomes one of the most prominent solutions for routing optimization in SDN [5]. Unlike conventional routing schemes, DRL-based routing algorithms establish agents to interact with network environments and explore optimal routing decisions and are thus capable of autonomously learning routing strategies without extensive prior knowledge or labeled data. Recent trends [6,7] emphasize

the transition from single-agent DRL (SADRL) to multi-agent DRL (MADRL) to enhance scalability and exploration capabilities.

However, it has been revealed in [8] that most existing DRL routing algorithms are constrained by a fixed topology and lack the robustness to handle topology changes. These DRL agents trained on one topology can only be deployed on the same topology without changes. Each time they are faced with a topology change, DRL agents require a long time for retraining, which is intolerable in real-world environments and significantly hinders the application of current DRL-based routing algorithms. As analyzed in [9], this limitation arises from the fact that DRL agents implemented by traditional neural networks (NNs) can only operate on matrices with fixed shapes corresponding to invariant topologies and struggle to extract graph-structured information to identify topology changes.

Fortunately, as a cutting-edge advancement in the field of machine learning, the Graph Neural Network (GNN) [10] is specifically designed to process graph-structured data with the message-passing mechanism. During message-passing, each node aggregates messages from its neighbors to update hidden features and is not constrained by the number of nodes or edges. Therefore, the GNN is able to operate on multiple topologies with varying structures. Considering the natural graph properties of network topologies, the GNN has demonstrated strong robustness and generalization capabilities in issues such as network modeling [8,9] and traffic prediction [11].

To our best knowledge, only a limited number of studies [12–20] have incorporated GNN into DRL-based routing algorithms. Replacing traditional NNs with GNNs, these works have successfully released DRL agents from the limitation of a fixed graph. However, it is notable that some GNN+DRL routing algorithms [13,14] still require retraining after topology changes or lack the evaluation of robustness. Other related GNN-enhanced works can be summarized in two aspects: (1) Most existing studies [15–17,19,20] focus on path-based flow routing, leading to the risk of link congestion in the face of elephant flows. In contrast, link-based packet routing enables each node to forward every incoming packet, capable of splitting flows from a fine-grained perspective to avoid one flow exceeding link capacity. (2) Most related research [12,15–18] is stuck in the SADRL framework, which suffers from the computational load of a single agent and struggles to explain the decision logic for each node and the relationships between nodes. On the contrary, MADRL enables independent decision-making for each node, enhancing the exploration capability, as well as reducing the computational burden of each agent. Regrettably, there is no algorithm leveraging the GNN to handle topology changes tailored for MADRL packet routing scenarios, which motivates us to fill this gap.

The second motivation we concentrate on is the design of the GNN for multiple agents to perceive traffic conditions within MADRL packet routing scenarios. With the goal of enhancing robustness against topology changes, DRL agents are encouraged to learn a universal packet routing strategy irrelevant to specific topologies, which lies exactly in the mapping from surrounding traffic of a node to the decision of which neighbor to forward packets to. This demands each node perceive adequate information for decision-making, especially detecting the links with potential congestion. A practical challenge in networks is that congestion information is likely to take a long distance of several hops to reach the decision-making node. However, almost all existing GNN+DRL routing algorithms only utilize basic GNNs like the Message-Passing Neural Network (MPNN) [21], where a single layer of the GNN can only perceive information from one-hop neighborhoods. To obtain the long-range congestion information, researchers have to stack multiple GNN layers, which will lead to the issue of over-smoothing [22], meaning that all node features become indistinguishable and the model loses the representational capability. Fortunately, the Graph Transformer [23,24], a recent advance in the GNN community, has been successfully

proposed to retain critical information during long-distance message-passing without suffering from over-smoothing, benefiting from designs such as the Transformer attention mechanism. It is a promising idea to leverage the power of the Graph Transformer to enhance long-range perception for routing optimization, which has never been mentioned in related works.

In this paper, we propose a novel algorithm, Graph Transformer Star Routing (GTSR), to enhance the robustness of intra-domain packet routing in SDNs. Compared to related works, GTSR integrates the GNN with MADRL for packet routing scenarios involving topology changes, emphasizing the perception of traffic at each node to autonomously optimize routing strategies. Dedicated to handling topology changes and addressing the above issues in related studies of inadequate designs of GNNs, we introduce the advanced Graph Transformer for processing different graphs after changes and carefully design the modules within the Graph Transformer for long-range perception. Specifically, we construct a global message-passing mechanism based on Transformer attention to update the hidden node features, during which a virtual global node called the star node is established to serve as the shortcut between distant nodes. In this manner, we only need to build one GNN layer, thus mitigating over-smoothing caused by deep layers of GNNs. Subsequently, multiple agents at nodes independently calculate routing decisions in parallel based on hidden features within the neighborhood. In this process, we design a path-based readout method, enhancing the ability of agents to detect information along multi-hop reachable paths for perceiving potential congestion. Furthermore, we propose a MADRL cooperation mechanism based on federated learning, facilitating the learning of general perception policies among agents while reducing computational overhead. To evaluate the performance and robustness of the proposed algorithm, we implement an SDN simulation environment involving intra-domain packet routing scenarios and conduct extensive experiments on multiple real-world network topologies. The experimental results demonstrate that GTSR is able to take a single topology for training and adapt to unseen topology changes without retraining, consistently maintaining superior QoS in terms of end-to-end (E2E) latency and packet loss rate compared to all baselines.

In summary, the main contributions of this paper include:

- We integrate a GNN with MADRL for packet routing in SDNs, enhancing robustness against topology changes.
- We establish a virtual star node and a Transformer-based global message-passing mechanism, enabling nodes to perceive long-distance traffic information.
- We propose a path-based readout method, improving the ability of nodes to accurately detect potential congestion along multi-hop paths.
- We design a MADRL cooperation mechanism to facilitate the learning of general perception strategies and reduce the amount of computation.
- We build an SDN simulation environment and conduct extensive experiments, demonstrating the superior robustness of the proposed algorithm compared to all baseline algorithms.

2. Related Work

With the explosive growth of network scale and traffic, traditional routing algorithms such as OSPF [1] and ECMP [2] have exposed deficiencies in the static strategy of only selecting shortest paths for transmission. SDN [3] has been proposed to decouple the control plane from the data plane, allowing researchers to focus on designing intelligent and flexible routing algorithms from a global perspective. To construct SDN simulation environments, OMNeT++ [25] is one of the most commonly used modular platforms excelling at conveniently simulating routing policies and topology changes in networks. As

a branch of deep learning, DRL has demonstrated outstanding performance for complex decision-making problems [4]. Considering routing as a series of decision-making processes, incorporating DRL into routing optimization has become one of the most promising directions in the field of networking [5]. Stampa et al. [26] applied DRL to optimize routing in SDN for the first time, while Valadarsky et al. [27] provided a comprehensive discussion of the DRL paradigm for routing. Recent trends [6,7] of DRL routing concentrate on replacing SADRL with MADRL to enhance exploration capabilities and disassemble computational tasks to multiple agents thus alleviating the burden on a single agent. It is also a hot topic to explore MADRL cooperation mechanisms, such as utilizing federated learning [28] to reduce overhead and facilitate the learning of general strategies.

However, DRL routing algorithms encounter challenges when applied to real-world scenarios. Rusek et al. [8] pointed out that most DRL routing algorithms are limited to a fixed topology due to the implementation by traditional NNs, which fail to handle matrices of varying shapes corresponding to topology changes and struggle to explicitly extract the relationships within a graph. Fortunately, GNNs [10] specialize in handling graph-structured data utilizing the message-passing mechanism, which is not constrained by the number of nodes or edges. Since networks are inherently graphs, GNN has demonstrated powerful generalization capabilities for several networking issues [9,11]. However, basic variants of GNNs like MPNN [21] only perform message-passing within a small range of neighborhoods, suffering from over-smoothing when stacking deep layers. Over-smoothing [22] refers to the phenomenon that node representations become indistinguishable after multiple GNN layers, ultimately leading to the loss of discriminative power. To address this issue, the Graph Transformer [23,24] has been successfully proposed to leverage Transformer attention mechanisms and cross-layer connections to retain crucial information during long-distance message-passing, demonstrating superior capacities of extracting features and mitigating over-smoothing.

To our best knowledge, only a small number of studies have incorporated GNNs into DRL-based routing algorithms to enhance robustness in handling topology changes, as presented in Table 1. In 2021, GDDR [12] introduced an MPNN into SADRL for the first time to split traffic with the goal of minimizing link congestion. GraphNET [13] combined an MPNN with SADRL to select the next hop for packets in SDN. DGATR [14] integrated a Graph Attention Network (GAT) into MADRL for packet routing. These early studies explored the potential of GNNs for DRL routing, but they lacked detailed evaluation of robustness on unseen topologies or still required retraining after changes. In 2022, Almasan et al. [15] utilized an MPNN and SADRL to select an optimal path for each flow, which inspired subsequent research [16,17] on SADRL path-based flow routing. In 2023, GAPPO [18] integrated a GAT into SADRL to compute link weights for packet forwarding. Bhavanasi et al. [19] leveraged a Graph Convolutional Network (GCN) and MADRL to avoid flow conflicts. MAGNNETO [20] introduced an MPNN into MADRL for optimizing link weights in traditional routing protocols.

As summarized in Table 1, most GNN+DRL routing algorithms focused on flow routing rather than fine-grained packet routing, or remained within the SADRL framework rather than scalable MADRL, which contributes to our first motivation to introduce GNNs into scenarios involving both MADRL and packet routing. Secondly, it is notable that all these studies only employed basic GNNs for message-passing within a limited range of neighborhoods, suffering from the risk of over-smoothing when stacking deep layers of the GNN for long-distance message-passing. The inadequacy of designs for GNNs in related works motivates us to strengthen the power of GNNs and leverage advanced Graph Transformers to enable long-range perception of traffic for learning routing strategies.

Table 1. Comparison of related GNN-enhanced DRL routing algorithms.

Reference	Routing Type	DRL Architecture	GNN Model	Superior Robustness	Shortcomings	Common Shortcoming
[12]	Flow	SADRL	MPNN	No	Failing to distinctly outperform baselines after topology changes	Only utilizing basic GNNs, suffering from the risk of over-smoothing and the deficiency in feature extraction capabilities during long-range message-passing
[13]	Packet	SADRL	MPNN	No	Requiring retraining after topology changes	
[14]	Packet	MADRL	GAT	No	Lacking evaluation of robustness against topology changes	
[15]	Flow	SADRL	MPNN	Yes	Selecting one path for each flow, only tailored for path-based flow routing	
[16]	Flow	SADRL	MPNN	Yes	Only tailored for path-based flow routing, constrained to a single agent	
[17]	Flow	SADRL	MPNN	Yes	Only tailored for path-based flow routing, constrained to a single agent	
[18]	Packet	SADRL	GAT	Yes	Constrained to a single agent, only evaluating for simple traffic	
[19]	Flow	MADRL	GCN	Yes	Only tailored for path-based flow routing	
[20]	Flow	MADRL	MPNN	Yes	Only tailored for path-based flow routing	

3. System Model and DRL Formulation

3.1. System Model

In this paper, we mainly consider intra-domain routing within an autonomous system (AS), which refers to a network consisting of a group of routers and is managed through Interior Gateway Protocol (IGP) for routing among these routers. The network is represented as $\mathcal{G}(\mathcal{N}, \mathcal{E})$, where $\mathcal{N}$ denotes the set of N nodes representing communication entities, and $\mathcal{E}$ denotes the set of E edges representing communication links. A node indexed by i is denoted as n_i , while the set of its one-hop and two-hop neighbors are represented as $\mathcal{N}_{nei_i}^{(1)}$ and $\mathcal{N}_{nei_i}^{(2)}$, respectively. We utilize e_{ij} to denote the link between nodes n_i and n_j . The link capacity c_{ij} describes the maximum traffic that the link e_{ij} can accommodate. $\mathcal{G}$ is regarded as an undirected and connected graph, meaning that all links are full duplex and there is at least one path between any pair of nodes. In network $\mathcal{G}$, there are multiple flows to be transmitted, each of which needs to be transferred from the source node n_{src} through several relay nodes to the destination node n_{dst} at a certain rate in *Mbps*. The routing algorithm needs to deliver all traffic to the destination with the goal of minimizing E2E delay and packet loss rate. It is known that the traffic on a link exceeding the link capacity will lead to queuing and a sharp increase in E2E delay. Packets that fail to reach their destination within the specified time will be discarded. Therefore, it is important for routing algorithms to identify and avoid link congestion for the objective of providing high QoS.

SDN is utilized to support the design of intelligent routing algorithms as shown in Figure 1. SDN decouples the data plane and the control plane, where the data plane focuses on executing practical forwarding operations with routing devices, while the control plane focuses on computing routing policies based on information collected from the data plane without the concern for detailed operational processes.

In the data plane, we employ probabilistic packet routing as the underlying routing protocol. Specifically, each node forwards each incoming packet to one of its neighboring nodes based on the weights of neighboring links as probabilities. Links with higher weights will be allocated more packets, while lower weights mean fewer packets. In this manner,

traffic is split from a fine-grained perspective, thus effectively avoiding link congestion caused by a single elephant flow and conflicts between flows.

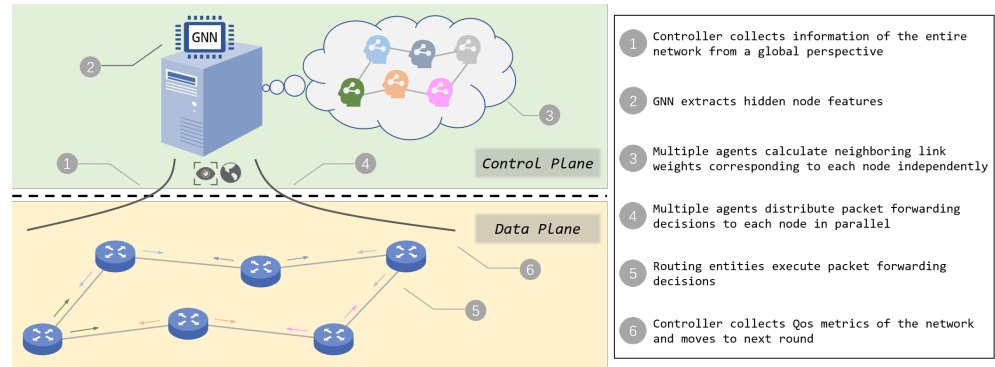


Figure 1. Architecture of SDN and operation process with multiple agents.

In the control plane, a controller is established for collecting all information within the entire network from a global perspective, subsequently calculating link weights and deploying forwarding decisions to the data plane for execution. To optimize routing strategies, we train multiple agents in the control plane in parallel, each of which corresponds to a node router in the data plane and fulfills the functions of planning, memory, tools, and action independently. The SDN architecture and operation process are illustrated in Figure 1.

3.2. DRL Formulation

We model routing as a Markov Decision Process (MDP) and utilize DRL to train agents for routing optimization. Specifically, we adopt a MADRL architecture, where each node corresponds to an independent agent. Each agent takes actions based on the observed states and receives rewards returned from the environment, aiming to optimize its strategies for maximizing cumulative rewards. For probabilistic packet routing in SDNs, we take the agent A_i corresponding to node n_i as an example and define the DRL formulation of state, action, and reward as follows:

State: We define the state s_i of A_i as the global traffic condition s_G of the network, including the traffic matrix (TM) and destination information. The TM is an $N \times N$ square matrix recording the traffic distribution across the network, where each element t_{ij} represents the traffic on link e_{ij} . The total traffic TM_{all} can be divided into N different TMs based on N distinct destinations, and the routing algorithm needs to make corresponding forwarding decisions for different destinations. For simplicity, we only describe the process of handling traffic destined for a certain node n_{dst} , while this process can be directly extended to all other destinations. Therefore, s_i consists of the TM for total traffic, the TM for destination n_{dst} , and the information of destination as:

$$s_i = s_G = \{TM_{all}, TM_{dst}, n_{dst}\}. \quad (1)$$

Action: We define the action a_i of A_i as assigning weights to the neighboring links. For traffic destined for n_{dst} , node n_i randomly selects one of its one-hop neighbors from $\mathcal{N}_{nei-i}^{(1)}$ to forward each packet based on the link weights as probabilities. a_i is a vector of length $N_{nei-i}^{(1)}$ as:

$$a_i = [w_{i1}, \dots, w_{iN_{nei-i}^{(1)}}] \in \mathbb{R}^{N_{nei-i}^{(1)}}. \quad (2)$$

Reward: We define the reward r_i of A_i as the global QoS metric r_G calculated by the integration of the average E2E delay and packet loss rate of the network. For each packet, the E2E delay is the difference between the arrival time and the sending time. Packets

that exceed the pre-defined survival time T_{sur} are discarded. Combining the actual delay of arrived packets and the time theoretically consumed by lost packets, we obtain the equivalent delay d_{eq} for the network as:

$$d_{eq} = \frac{1}{P} \sum_p (t_{p_arr} - t_{p_send}) + T_{sur} \times \frac{loss_rate}{1 - loss_rate} \quad (3)$$

$$r_i = r_G = -d_{eq},$$

where the reward is defined as the negative of the equivalent delay, as the goal of maximizing the reward corresponds to minimizing the delay and packet loss rate.

4. Method

After obtaining the system model and DRL formulation, GTSR leverages the global message-passing and path-based readout to map states to multi-agent actions, as well as establishing a MADRL cooperation mechanism to facilitate the learning of the general perception method.

4.1. Global Message-Passing with Graph Transformer and Star Node

We utilize a Graph Transformer and additionally design a global message-passing mechanism with a star node to deliver crucial information during long-range perception, tackling the deficiencies of basic GNNs like the MPNN.

Basic GNNs only enable nodes to aggregate messages within the one-hop neighborhood and lack efficient approaches to distinguish the importance of neighbors. Consider a scenario at the top of Figure 2, where node n_i needs to perceive the congestion information of distant node n_{far} and thereby reduces the weights of the left-side neighboring link. Basic GNNs need to stack at least 3 layers to cover the distance of 3 hops from n_{far} to n_i . However, in this manner, distant messages have to be transmitted through the hidden features of hidden features, leading to the loss of information. Furthermore, deeper GNN will suffer from over-smoothing, where all hidden node features converge to the same value after excessive information exchange, ultimately leading to the loss of representational ability.

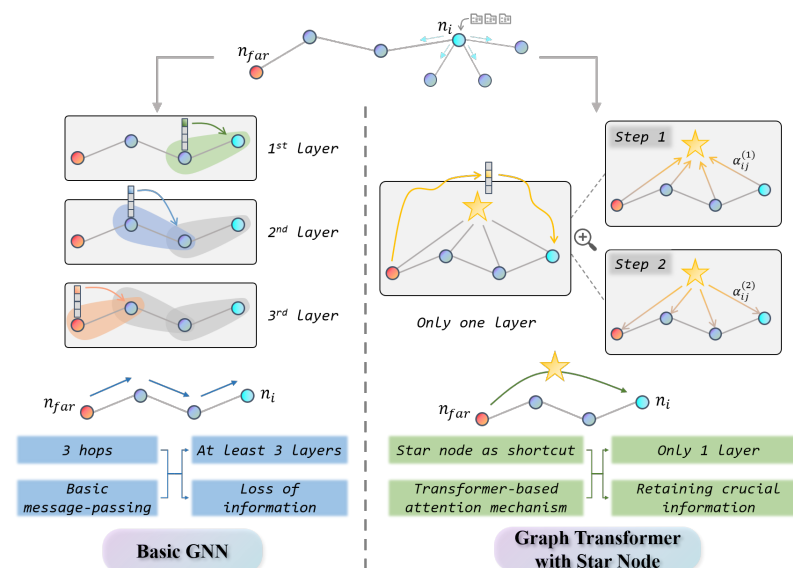


Figure 2. Comparison of basic GNN and Graph Transformer with star node during long-range message-passing. To make packet forwarding decisions, node n_i (colored cyan) needs to receive messages from distant congested node n_{far} (colored red). Basic GNN has to stack at least three layers to obtain a perception range of 3-hop neighborhood in a hop-by-hop manner (green-blue-red), while we only need one layer of Graph Transformer with the star node as a shortcut (colored golden).

In this paper, we propose the Graph Transformer with star node, which only needs one layer for long-range perception, thus mitigating over-smoothing. Compared to basic GNNs, the emphasis and advantages of the proposed method include two aspects:

- (1) Graph Transformer employs the Transformer attention mechanism to enable nodes to selectively aggregate information from the neighborhood, ensuring that crucial information is retained during message-passing.
- (2) The designed star node is a global virtual node connecting to all nodes in the graph, which serves as the bridge between any pair of nodes. The operation of the star node consists of two steps. Firstly, the star node aggregates the features of all nodes based on Transformer attention. Secondly, the star node participates in the original message-passing of each node. In this manner, the star node creates a shortcut for long-range perception, where the information of distant nodes is able to be embedded in the hidden feature of the star node and then directly transmitted to any other node.

Combining the two aspects mentioned above, we only need to construct one layer of the Graph Transformer to enable long-distance message-passing, addressing the issues of over-smoothing and information loss caused by deep layers of basic GNNs. The detailed description of the proposed global message-passing is as follows.

Firstly, we convert the global state s_G into the set of initial node features $X = \{x_1, \dots, x_N\}$. Specifically, the initial feature vector x_i for node n_i consists of 7 elements. The first three elements represent the maximum, minimum, and sum of the traffic flowing out of node n_i , while the next three elements represent the traffic flowing into n_i . The final element indicates the minimum hops from n_i to the destination n_{dst} . This design standardizes the length of initial feature vectors for all nodes and eliminates the influence of node indexing.

After obtaining the initial node feature set X , we perform global message-passing to update X into the hidden node feature set $X' = \{x'_1, \dots, x'_N\}$. As shown in Figure 3, global message-passing is divided into two steps.

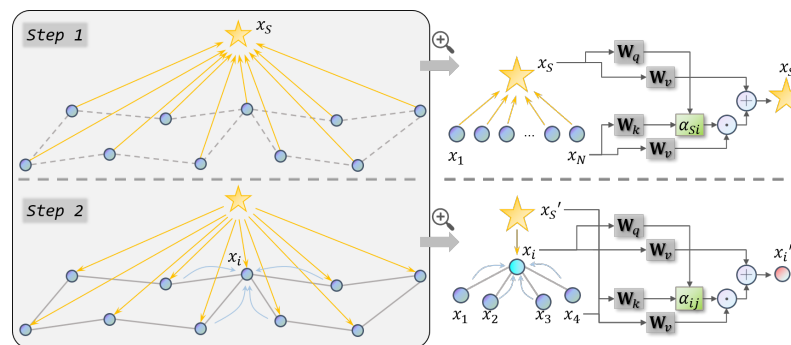


Figure 3. Detailed process of global message-passing with Transformer attention and star node. The star node (colored golden) first aggregates information from all nodes (golden arrows from x_i to x_s) and then participates in message-passing of all nodes (golden arrows from x_s to x_i).

Step 1: We establish a star node denoted as n_s and aggregate the features of all nodes to update the star node feature from x_s to x'_s . The initial feature of the star node is also a vector of length 7, but all elements are set to 1. The hidden feature x'_s is a vector of length H . In this step, we update only the star node, excluding other original nodes, so the entire graph can be represented as $\mathcal{N}_{step1} = \mathcal{N} \cup \{n_s\}$ and $\mathcal{E}_{step1} = \{e_{1s}, e_{2s}, \dots, e_{Ns}\}$. For the star node n_s and the original nodes $n_j \in \mathcal{N}$, we introduce the Transformer attention mechanism into feature aggregation to obtain x'_s as:

$$\begin{aligned} q_S^{(S)} &= W_q^{(S)} x_S, \quad k_j^{(S)} = W_k^{(S)} x_j, \quad \alpha_{Sj}^{(S)} = \text{softmax}\left(\frac{q_S^{(S)T} k_j^{(S)}}{\sqrt{H}}\right), \quad \text{for } n_j \in \mathcal{N} \\ v_S^{(S)} &= W_v^{(S)} x_S, \quad v_j^{(S)} = W_v^{(S)} x_j, \quad x'_S = v_S^{(S)} + \sum_{n_j \in \mathcal{N}} \alpha_{Sj}^{(S)} v_j^{(S)}, \end{aligned} \quad (4)$$

where each $W_{-}^{(S)}$ is a learnable matrix implemented by fully connected (FC) layers to map the initial features into a vector of length H in the high-dimensional space. The query matrix $W_q^{(S)}$ for star node and the key matrix $W_k^{(S)}$ for original nodes are utilized to obtain the attention coefficients α_{Sj} , which represent the importance of each original node. These coefficients are subsequently employed as weights in the weighted sum of all value vectors.

Step 2: The updated star node participates in the message-passing of all original nodes $n_i \in \mathcal{N}$ to update the initial feature x_i to the hidden feature x'_i . In this step, the star node only participates in the message-passing of all original nodes without updating itself, so the entire graph can be represented as $\mathcal{N}_{step2} = \mathcal{N} \cup \{n_S\}$ and $\mathcal{E}_{step2} = \mathcal{E} \cup \{e_{S1}, e_{S2}, \dots, e_{SN}\}$. Taking the original node n_i as an example, we obtain its new one-hop neighborhood denoted as $\mathcal{N}_{nei_i}^{(S)} = \mathcal{N}_{nei_i}^{(1)} \cup \{n_S\}$. Then, we aggregate features from $n_j \in \mathcal{N}_{nei_i}^{(S)}$ to update the feature of n_i to x'_i as:

$$\begin{aligned} q_i &= W_q x_i, \quad k_j = W_k x_j, \quad k_S = W_k x'_S, \quad \alpha_{ij} = \text{softmax}\left(\frac{q_i^T k_j}{\sqrt{H}}\right) \quad \text{for } n_j \in \mathcal{N}_{nei_i}^{(S)} \\ v_i &= W_v x_i, \quad v_j = W_v x_j, \quad v_S = W_v x'_S, \quad x'_i = v_i + \sum_{n_j \in \mathcal{N}_{nei_i}^{(S)}} \alpha_{ij} v_j, \end{aligned} \quad (5)$$

where W_{-} is another set of learnable matrices distinct from $W_{-}^{(S)}$ in the first step. Applying the above process simultaneously to all original nodes, we obtain the hidden node feature set $X' = \{x'_1, \dots, x'_N\}$.

From the entire design of the proposed global message-passing with star node, it is observed that Transformer attention mechanism equips nodes with the ability to distinguish neighbors and extract crucial information during message-passing. Strengthening the advantages of the Graph Transformer, the star node further facilitates the transmission of long-distance information. It is notable that we need to additionally build one node and N links, which will somewhat increase the computational overhead within a GNN layer. Fortunately, we only need to construct one layer of the Graph Transformer rather than multiple layers of basic GNNs, thus mitigating the increase in computation.

4.2. Path-Based Readout for Multiple Agents

After global message-passing, each agent corresponding to each node will readout the hidden node features within its neighborhood to the one-hop neighboring link weights.

In baseline algorithms, most readout methods directly utilize one feature vector of a one-hop neighbor or the concatenation of the two feature vectors of the one-hop neighbor and the central node to calculate the corresponding neighboring link weight. However, these readout methods constrain agents to a limited range of one-hop neighborhoods, hindering agents from exploring distant node congestion information to decide whether to reduce traffic in a particular direction.

Therefore, in this paper, we are dedicated to expanding the perceptive field of agents during readout for long-range perception and routing decision-making. Specifically, a path-based readout method is proposed, where each node is able to focus on detecting potential congestion of all reachable paths through the concatenated vectors representing these paths. The designs and advantages of the path-based readout include:

- (1) During the expansion to multi-hop neighborhoods, a practical problem is that the numbers of nodes from different directions are probably variable, leading to concatenated vectors for certain directions with different lengths. To address this issue,

path-based readout focuses on concatenating node feature vectors along all paths from the central node, since paths with identical hops share the same number of nodes.

- (2) Compared to operations such as averaging and summing, concatenating retains all information along paths, thus facilitating the central node to detect potential congestion of certain paths and subsequently decrease the corresponding link weights.

As illustrated in Figure 4, we will describe the detailed path-based readout method in a scenario involving two-hop paths, which can be extended to any other multi-hop paths. For agent A_i located at node n_i , we first create a local subgraph $\mathcal{G}_{sub_i}$ centered at n_i , where the node set $\mathcal{N}_{sub_i} = n_i \cup \mathcal{N}_{nei_i}^{(1)} \cup \mathcal{N}_{nei_i}^{(2)}$. All node features in $\mathcal{G}_{sub_i}$ are the hidden features after global message-passing. Next, we extract all two-hop paths starting from n_i . In the case shown in Figure 4, n_i has three one-hop neighbors and four two-hop neighbors. The four paths from n_i are represented as $path_1 : n_i \rightarrow n_1 \rightarrow n_{1-1}$, $path_2 : n_i \rightarrow n_2 \rightarrow n_{2-1}$, $path_3 : n_i \rightarrow n_3 \rightarrow n_{3-1}$, $path_4 : n_i \rightarrow n_3 \rightarrow n_{3-2}$, where $path_3$ and $path_4$ both pass through the one-hop neighboring node n_3 . To obtain the representation vector $p \in \mathbb{R}^{3H}$ for each path, we concatenate the hidden features of the nodes along each path as:

$$p_1 = x'_i || x'_1 || x'_{1-1}, \quad p_2 = x'_i || x'_2 || x'_{2-1}, \quad p_3 = x'_i || x'_3 || x'_{3-1}, \quad p_4 = x'_i || x'_3 || x'_{3-2}. \quad (6)$$

Then, we utilize a learnable matrix $W_{p1} \in \mathbb{R}^{2H \times 3H}$ to extract the hidden path vector p' as:

$$p'_1 = W_{p1}p_1, \quad p'_2 = W_{p1}p_2, \quad p'_3 = W_{p1}p_3, \quad p'_4 = W_{p1}p_4. \quad (7)$$

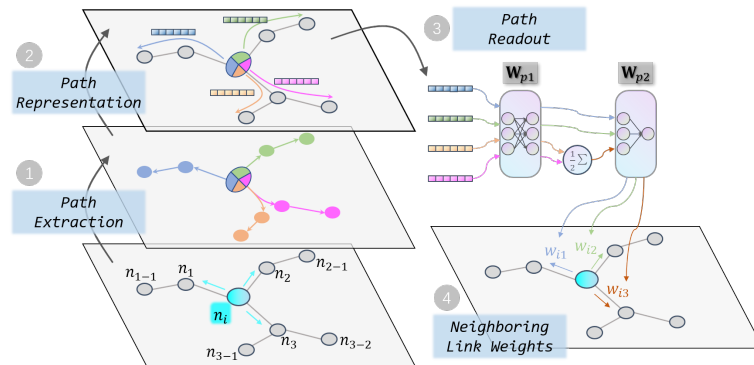


Figure 4. Path-based readout for multiple agents. Node n_i has four reachable tow-hop paths (colored blue, green, yellow, red) and needs to calculate three neighboring link weights (colored blue, green, brown). With path-base readout, n_i is able to detect all information along reachable paths and perceive the direction of potential congestion.

It is notable that the one-hop neighboring node n_3 has two branching paths, while we only need one neighboring link weight w_{i3} . Therefore, we choose to merge the hidden features of all paths passing through the same one-hop neighboring node, which are ultimately mapped to a single value by $W_{p2} \in \mathbb{R}^{2H}$ as:

$$w_{i1} = W_{p2}p'_1, \quad w_{i2} = W_{p2}p'_2, \quad w_{i3} = W_{p2}\left(\frac{1}{2}(p'_3 + p'_4)\right). \quad (8)$$

In this way, we obtain the weights of all one-hop neighboring links for node n_i as the action of agent A_i . Parallelizing the above operations across all nodes, we obtain the routing decisions for the entire network.

In addition, we also need to read out an additional real number as the state value $V(s_i)$, which will be used in agent training. $V(s_i)$ evaluates the state of A_i , so we take the

average of representation vectors of reachable paths from n_i and employ another learnable matrix $W_{vs} \in \mathbb{R}^{3H}$ to map it to a real number as:

$$V(s_i) = W_{vs} \left(\frac{1}{4} (p_1 + p_2 + p_3 + p_4) \right). \quad (9)$$

4.3. MADRL Operation and Cooperation

In this section, we carefully design the MADRL operation and cooperation mechanism to combine the above proposed global message-passing with the readout method. Compared to SADRL, MADRL allows each agent to make decisions independently, which significantly enhances the exploration capability of the routing algorithm and aligns closely with the characteristics of packet routing. However, the limitation of MADRL arises during message-passing, where each agent is not allowed to expand its perceptive range indefinitely; otherwise, the total amount of computation will explode.

Unfortunately, there are no baseline studies proposing an MADRL cooperation mechanism to combine the advantages of global message-passing and independent decision-making. The only GNN+MADRL-based algorithm involving MADRL cooperation is [14], where federated learning is utilized for agents to share all parameters in NNs and obtain a set of averaged NN parameters. However, simply sharing the entire NNs actually undermines the capabilities of independent decision-making for multiple agents.

In this paper, we refine the scope of federated learning and propose an MADRL cooperation mechanism, addressing the limitation of MADRL during global message-passing, as well as retaining its strength of independent decision-making. The motivations and designs of the proposed method include:

- (1) We leverage federated learning to average only the parameters in GNNs among agents for global message-passing and preserve the independent updating of FC NNs for the readout of each agent. In this architecture, the shared GNN module consisting of averaged parameters is actually a general version for all agents and facilitates the learning of the universal rule of message-passing.
- (2) Without federated learning for parameter sharing, each agent needs to maintain a distinct GNN to process the entire graph, while only a few local node features are employed for readout, leading to extensive redundant calculations. Benefiting from the proposed cooperation mechanism, all agents share one set of GNN parameters and their inputs are also the same graph. Therefore, we just need to perform the global message-passing only once and enable each agent to directly fetch the required node features, thus reducing a large amount of computation.

We will introduce the detailed MADRL operation and cooperation as follows.

Multi-Agent Operation: The MADRL operation is displayed vertically in Figure 5. The global state s_G is distributed to all agents in parallel. Each agent A_i takes the full graph as input and utilizes a GNN to perform global message-passing for feature extraction. Subsequently, A_i only extracts the hidden features within its local subgraph $\mathcal{G}_{sub_i}$ and reads out the one-hop neighboring link weights as the action a_i .

Specifically, each agent is implemented by Proximal Policy Optimization (PPO) [29], a classic DRL algorithm consisting of an actor and a critic. The actor maps the state s to the action a , while the critic maps the state s to the state value $V(s)$. All NNs of an agent are denoted as θ , which can be split into the global message-passing module θ_{mp} and the local readout module θ_{ro} . To update θ , we use the loss functions $L^{ACT}(\theta)$ and $L^{CRI}(\theta)$ to perform back-propagation in the actor and critic, respectively:

$$L^{ACT}(\theta) = \min(ra(\theta)Ad, \text{clip}(ra(\theta), 1 - \epsilon, 1 + \epsilon)Ad), \quad L^{CRI}(\theta) = (V_\theta(s) - \sum \gamma r)^2, \quad (10)$$

where ra represents the ratio of action distributions before and after updating, Ad is the advantage estimator [29], and γ is the reward decay factor. The final loss is $L^{PPO}(\theta) = L^{ACT}(\theta) - L^{CRI}(\theta) + S[\pi](s)$, where $S[\pi]$ is the entropy reward to encourage exploration.

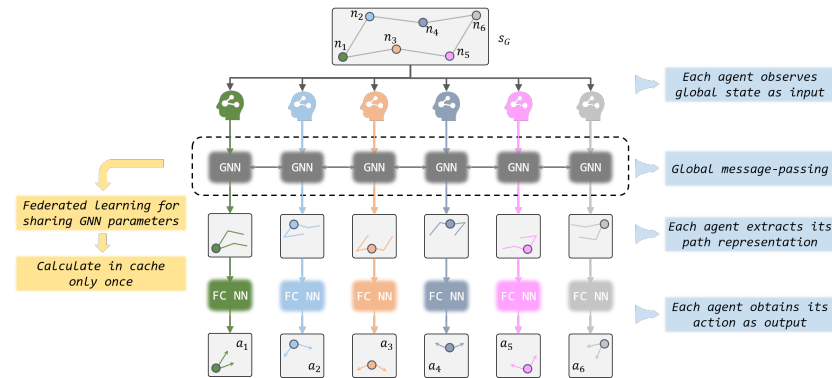


Figure 5. Multi-agent operation and cooperation. All agents (with different colors) will take the entire graph as input, while the global message-passing will be executed only once, since the GNN parameters of all agents are shared based on federated learning (all colored dark gray). Then each agent utilizes hidden features within its subgraph and its own FC NN (with the corresponding color) for routing decision-making independently.

Multi-Agent Cooperation: In the above MADRL operation, it is evident that agents need to repeat global message-passing, while only nodes within the local subgraph are employed for decision-making. Therefore, we design a MADRL cooperation mechanism based on the federated learning paradigm, aiming to reduce redundant computation and assist all agents in learning a universal perception strategy. As displayed horizontally in Figure 5, agents only share GNN parameters θ_{mp} of the message-passing module, excluding the FC NN parameters θ_{ro} of the readout module, thus maintaining the independent decision-making capability of multiple agents. Specifically, during updating from the t^{th} round to the $(t + 1)^{th}$ round, each agent A_i calculates its temporary parameters θ'_{i_mp} of the message-passing module based on gradients and the loss function as:

$$\theta'_{i_mp} = \theta_{i_mp}^{(t)} - \nabla_{\theta_{i_mp}} L^{PPO}(\theta_i). \quad (11)$$

Then, we calculate the average of all θ'_{mp} from all agents as the updated parameters as:

$$\theta_{i_mp}^{(t+1)} = \frac{1}{N} \sum_{n_i \in \mathcal{N}} \theta'_{i_mp}. \quad (12)$$

For the readout module, each agent updates its θ_{ro} independently as:

$$\theta_{i_ro}^{(t+1)} = \theta_{i_ro}^{(t)} - \nabla_{\theta_{i_ro}} L^{PPO}(\theta_i). \quad (13)$$

Furthermore, since the inputs and outputs of the global message-passing module for all agents are identical, the computation within the GNN is loaded into the cache and executed only once, thus reducing the total amount of computation.

In summary, the proposed MADRL cooperation mechanism accommodates both the global message-passing and the independent decision-making for multiple agents. The computational complexity of each agent indeed expands from the local subgraph to the whole graph, whereas we design the federated learning-based parameter sharing so that global message-passing only needs to be executed once for all agents, thus decreasing the computation amount. A potential limitation is the operation of directly exchanging parameters, which is likely to interfere with the exploration direction of those learned

agents in early stages of training. This operation can be replaced with sharing experience in future research, requiring the state space for all agents to be united into the same shape.

5. Experiments

5.1. Simulation Setup

To evaluate the performance and robustness of the proposed algorithm, we conduct extensive experiments, concentrating on assessing whether the algorithm trained on a single graph can maintain high QoS on unseen topologies after link failures without any reconfiguration or retraining. Specifically, we divide each experiment into the training phase and the testing phase. During training, the algorithm is only allowed to use an original network topology denoted as $\mathcal{G}_o$ to optimize routing policies with the objective of minimizing E2E delay and packet loss rate. Once the training is complete, all NN parameters of the algorithm are saved for subsequent testing. During testing, we randomly remove links from the graph to simulate link failures and obtain a series of faulty topologies denoted as $\{\mathcal{G}_d\}$. The algorithm loads the trained model to directly forward packets on $\mathcal{G}_d$, during which it is not allowed to use new testing experience for updating NN parameters. This is a challenging task considering that the routing algorithm has never encountered these faulty topologies before and needs to utilize the routing policy learned on $\mathcal{G}_o$ for transmitting data on unseen $\mathcal{G}_d$.

We utilize OMNeT++ [25] to construct an SDN simulation environment involving intra-domain packet routing scenarios. Compared to real-world environments, the simulation environment is convenient to build and modify, facilitating the exploration of different designs of routing algorithms. The algorithms trained in a simulation environment only require little overhead for fine-tuning to be applicable to complex real-world environments. For network topologies, we collect several real-world networks from the Topology Zoo dataset [30], as shown in Figure 6. All links are full duplex with a capacity c_{ij} of 1 Mbps, and the transmission delay on each link is 2 ms. Probabilistic packet routing is employed as the routing protocol. For traffic demands, we generate uniform traffic, meaning that each node sends data to all other nodes at a uniform rate in Mbps. The packet size is set to 512 B, and the packet survival time T_{sur} is 2 s, both of which are set to moderate values to balance the simulation speed and processing power of the routers. The step time is configured as 2 s, indicating that for every 2 s the environment returns TM, E2E delay, and the packet loss rate. In this study, we primarily focus on heavy-load traffic rather than light-load traffic, since the QoS of different algorithms is similar when there is no link congestion under light-load traffic. In contrast, under heavy-load traffic, algorithms need to precisely calculate routing decisions to avoid any congestion on all links; otherwise, packet queuing will lead to a dramatic increase in E2E delay and packet loss rate.

We leverage Pytorch and Torch_geometric [31] to implement GTSR. Specifically, we establish multiple agents in the control plane corresponding to each SDN router located at each node in the data plane. Then, we create independent processes in parallel for each pair of agents and routers to communicate. For the GNN, a single-layer Graph Transformer is constructed for global message-passing, while the number of hops during path-based readout is set to two-hop paths. Each learnable matrix W is implemented by an FC NN, and the length of the node hidden feature vector H is set to 16. For MADRL, the ϵ in the clip function and the reward discount factor γ are set to 0.2 and 0.6, respectively. Every 100 steps, the agents update for 10 epochs using the Adam optimizer with a learning rate of 0.001. Training on $\mathcal{G}_o$ lasts for 36,000 steps, while testing on each $\mathcal{G}_d$ lasts for 60 steps.

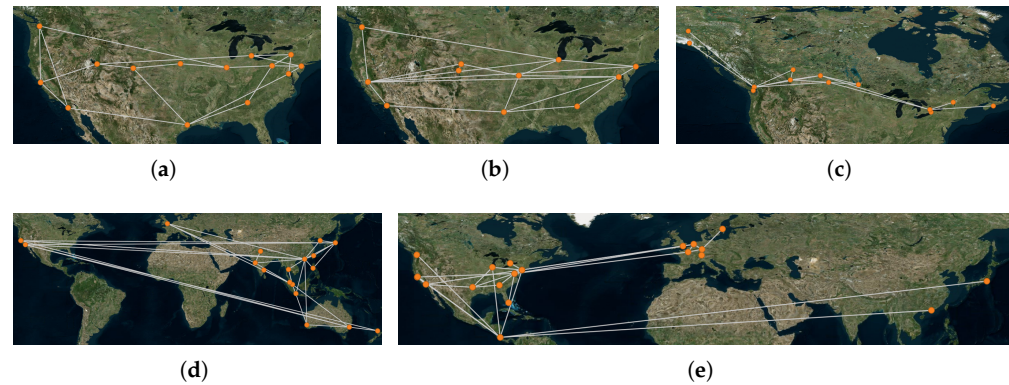


Figure 6. Real-world network topologies utilized in this paper: (a) Nsfnet. (b) Sprint. (c) Navigata. (d) BtAsiaPac. (e) HurricaneElectric.

For benchmarks, we utilize the following five algorithms:

- ECMP [2], a traditional multi-path routing algorithm that tends to fixedly distribute each flow to all equal-cost shortest paths between the source and the destination. Path cost is determined by number of hops.
- GAPPO [18], a previous packet routing algorithm that combines PPO and GAT within SADRL architecture. The initial node feature is the specific row of the TM after padding with zeros. The single agent establishes a three-layer GNN to directly calculate the weights of all links.
- DGATR [14], a packet routing algorithm integrating Deep Q-Network (DQN) with a GAT within MADRL architecture. Each agent observes the indices of the current node, neighboring nodes, and the destination node as the initial node feature and employs the GNN to determine the next hop for each packet.
- GTSR-MP-k, a cut-down version of the proposed GTSR. The only difference from GTSR is that the single-layer Graph Transformer for global message-passing is replaced by a traditional k-layer MPNN, which will reveal the impact of GNN layers.
- GTSR-RO-k, another cut-down version of the proposed GTSR, where the path-based readout method is replaced by a simple readout with k-hop neighborhoods. This variant will reveal the impact of different readout methods.

5.2. Analysis of Different GNN Layers

Considering the critical role of traffic perception for routing optimization, we first explore the impact of the perception range during global message-passing on the learning capability of routing algorithms. One layer of basic GNN aggregates messages within the one-hop neighborhood, while a k-layer GNN provides a perceptive field of k-hop neighborhoods. We replace the Graph Transformer in GTSR with a k-layer MPNN and obtain five GTSR-MP-k variants with $k = 1, 2, 4, 6, 8$. Nsfnet is employed as a case study, which consists of 14 nodes and 21 links. For uniform traffic, the sending rate of each flow is set to 0.04 Mbps. We train five GTSR-MP-k algorithms and GTSR for 36,000 steps on the original topology $\mathcal{G}_o$ of Nsfnet.

Figure 7 illustrates the curves of reward, E2E delay, and packet loss rate for all algorithms during training. The reward is the negative of the equivalent delay calculated by E2E delay and packet loss rate in Equation (3). GTSR-MP-1, the red curve in Figure 7, declines most rapidly during the first 3000 steps but encounters obstacles during the middle stage, ultimately converging as the last one. In contrast, the curves of two-layer and four-layer GNNs demonstrate that expanding the perception range enhances the ability to explore optimal routing strategies and accelerates the final convergence. However, indefinitely increasing the number of GNN layers will result in severe over-smoothing, meaning that

node features become too smooth and indistinguishable after excessive rounds of information exchanges. Stacking excessive GNN layers, GTSR-MP-8 suffers from over-smoothing and loses the representational capability for routing, ultimately failing to learn a routing strategy to achieve low E2E delay and packet loss rate.

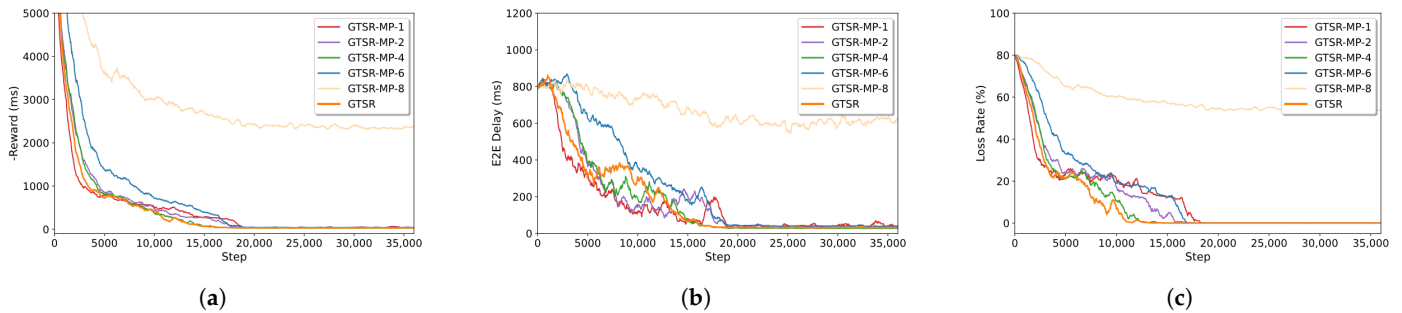


Figure 7. Training curves of six routing algorithms with different GNN layers: (a) Reward during training. (b) E2E delay during training. (c) Packet loss rate during training.

Figure 8 presents the final training results for algorithms with different numbers of GNN layers. In Figure 8a, we plot the average, maximum, and minimum equivalent delays during the last 100 steps of training. It is evident that increasing the perception field to a certain range improves the exploration of optimal routing strategies, while excessive GNN layers lead to over-smoothing and worse performance. Benefiting from the Graph Transformer with the star node, GTSR only utilizes one layer to enable long-range perception without the problem of over-smoothing caused by the deep layers of the GNN, finally converging to the lowest equivalent delay. Figure 8b illustrates the total runtime of training, which can be analyzed according to the computational complexity of different algorithms. A single layer of global message-passing maps the features of N nodes with the length of H to new hidden features with the length of H' . Each layer of GNN has a separate set of parameters, and all layers of the GNN will be executed sequentially. Therefore, the complexity of GTSR-MP- k with k -layer global message-passing is $O(kNHH')$. For GTSR, the single layer of the Graph Transformer with a star node can be regarded as one layer of global message-passing for $N + 1$ nodes. Therefore, the complexity of global message-passing in GTSR is $O((N + 1)HH')$. From Figure 8b, it is evident that the runtime of GTSR-MP- k increases linearly with the number of layers, while the runtime of GTSR lies between GTSR-MP-1 and GTSR-MP-2, which corresponds to the complexity. In summary, GTSR-MP-4 exhibits the best training performance among all baselines and will be selected for subsequent robustness evaluation. The proposed GTSR demonstrates a larger perception field than four-hop neighborhoods and obtains the lowest converged delay, even though it only requires a single layer of the GNN and fewer computational resources.

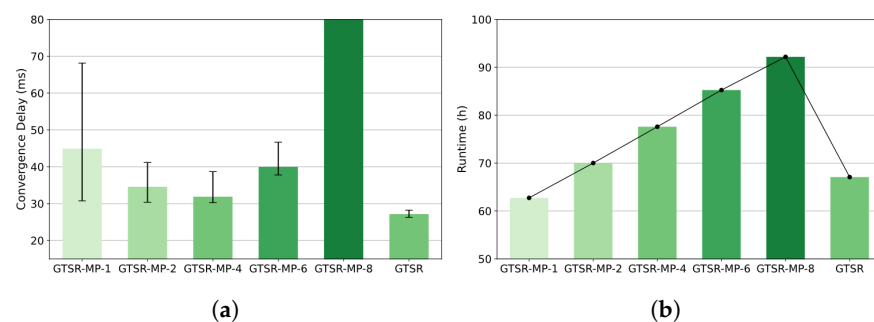


Figure 8. Training performance of six routing algorithms with different GNN layers: (a) Equivalent delay after convergence during the last 100 steps of training. (b) Runtime of the entire training.

5.3. Analysis of Different Readout Methods

In this section, we analyze the impact of different readout methods on learning capabilities of routing algorithms. Specifically, we design three baselines with distinct readout methods to compare with GTSR. Taking node n_i and its readout for one neighboring link e_{ij} as an example, GTSR-RO-0 only utilizes a single feature vector of n_i to compute the link weight w_{ij} . GTSR-RO-1 utilizes the concatenation of two feature vectors of n_i and n_j for computation. GTSR-RO-2 averages features of all nodes within the two-hop neighborhood in the direction of e_{ij} . Compared to GTSR-RO-2, GTSR also utilizes two-hop neighborhood information, while it focuses on paths and employs concatenation instead of average to retain more information for detecting congestion. We train the above four algorithms for 36,000 steps on the original topology $\mathcal{G}_o$ of Nsfnet under uniform traffic with a sending rate of 0.04 Mbps.

Figures 9 and 10 illustrate the training performance of the four algorithms on $\mathcal{G}_o$. In Figure 9, GTSR-RO-0 displays a relatively fast learning speed in the early stage, since the readout involves only one node and fewer parameters to be learned. However, it reveals a deficiency in learning capability during the middle and late stages due to the limited perception range, ultimately failing to converge to an E2E delay below 100 ms. Other algorithms increase the amount of information for the readout, successfully achieving the optimal strategy. Figure 10 depicts the converged equivalent latency and the runtime during training. In Figure 10a, GTSR-RO-0 performs the worst, while GTSR-RO-1 achieves better performance, as it detects more information for routing decision-making. It is interesting that GTSR-RO-2 under-performs GTSR-RO-1, even though it extends the perception range from one-hop to two-hop neighborhoods. The reason lies in the operation on neighborhood information, where GTSR-RO-2 employs the average rather than the concatenation of features, leading to the loss of crucial information and a poor learning capability. In contrast, GTSR addresses this issue with a path-based readout mechanism, which successfully extends the perception range and retains all information along paths through the operation of concatenating, ultimately achieving the lowest convergence latency.

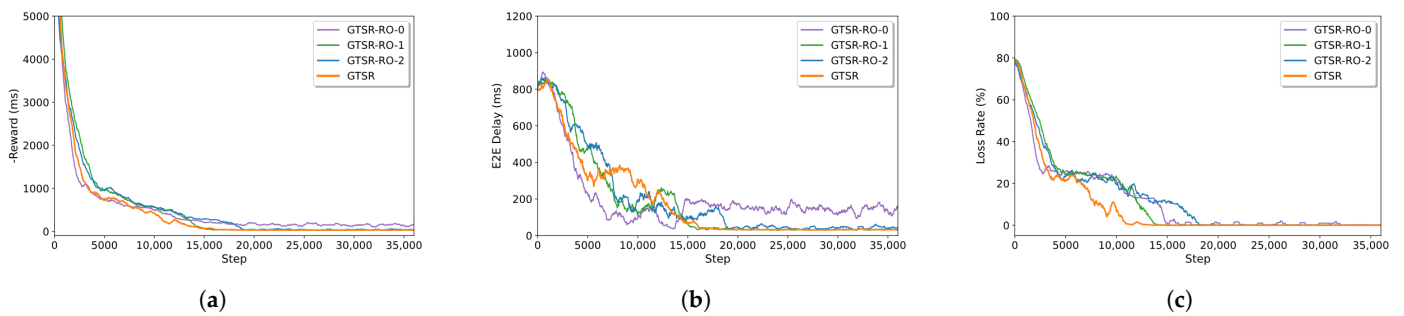


Figure 9. Training curves of four routing algorithms with different readout methods: (a) Reward during training. (b) E2E delay during training. (c) Packet loss rate during training.

Figure 10b presents the runtime of algorithms during training, reflecting the computational complexity of different readout methods. Note that each node has $N_{nei}^{(1)}$ one-hop neighbors and $N_{nei}^{(2)}$ two-hop neighbors. GTSR-RO-0 maps hidden features of length H to link weight vector of length $N_{nei}^{(1)}$, while GTSR-RO-1 uses concatenated vectors of length $2H$, leading to the complexity of $O(HN_{nei}^{(1)})$ and $O(2HN_{nei}^{(1)})$, respectively. GTSR-RO-2 calculates the average of $N_{nei}^{(1)} + N_{nei}^{(2)}$ node features of length H , with a complexity of $O(H(N_{nei}^{(1)} + N_{nei}^{(2)}))$. The computation of the proposed GTSR involves $N_{nei}^{(2)}$ paths represented by vectors of length $3H$, resulting in a complexity of $O(3HN_{nei}^{(2)})$. It is observed from Figure 10b that GTSR consumes higher runtime than baselines, since it expands the

perception range and the amount of information. However, these designs are necessary and effective for long-range perception, ultimately improving the performance of GTSR. In conclusion, GTSR-RO-1 achieves the best training performance among the baselines and is selected for the subsequent robustness evaluation. The proposed GTSR extends the perception range while retaining all path information for routing optimization. Despite the increase in complexity, GTSR enhances the learning capability and achieves the lowest convergence latency.

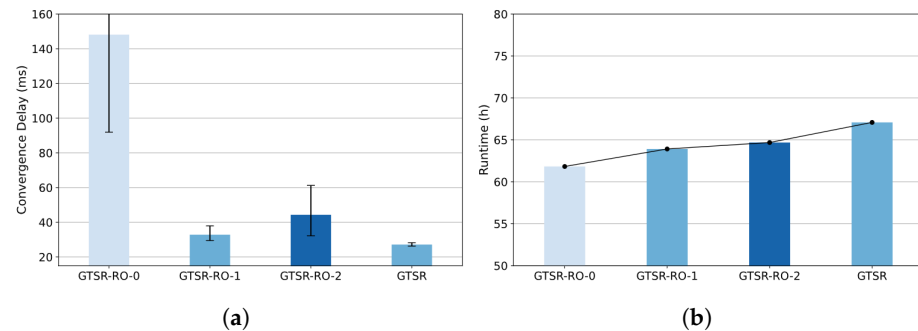


Figure 10. Training performance of four routing algorithms with different readout methods: (a) Equivalent delay after convergence during the last 100 steps of training. (b) Runtime of the entire training.

5.4. Evaluation on Unseen Topologies

In this section, we focus on robustness evaluation, thoroughly assessing the performance of GTSR and baseline algorithms on unseen topologies with link failures. Specifically, we employ Nsfnet as a case study and denote its original topology as $\mathcal{G}_o$. After randomly deleting links, we obtain a series of faulty topologies denoted as $\{\mathcal{G}_d\}$, comprising 21 $\mathcal{G}_{d1}$ with one link failure and 185 $\mathcal{G}_{d2}$ with two link failures. Note that faulty topologies that become disconnected after link failures will be excluded. All routing algorithms are trained on $\mathcal{G}_o$ for 36,000 steps with the objective of minimizing E2E latency and packet loss rate. In the next phase of testing, routing algorithms load their trained models to directly route on each unseen faulty topology $\{\mathcal{G}_d\}$ for 60 steps without retraining and updating NN parameters.

Figure 11 depicts the detailed equivalent delay d_{eq} of each algorithm during testing on the 21 faulty topologies $\mathcal{G}_d$ involving one link failure. Each 3D subplot in Figure 11 corresponds to an algorithm, with each line representing d_{eq} over 60 steps on a specific $\mathcal{G}_d$. A larger area between the curve and the xy-plane means a higher delay. It can be observed that the traditional routing algorithm ECMP maintains stable high delays on most $\mathcal{G}_d$, as it fixedly selects a few shortest paths without dynamically utilizing available links to avoid congestion. For DRL-based routing algorithms, GAPPO struggles to generalize to unseen topologies, as its single-agent architecture fails to distinguish actions for individual nodes, leading to over-fitting on $\mathcal{G}_o$ and poor generalization on $\mathcal{G}_d$. Other MADRL-based algorithms perform better than GAPPO, benefiting from independent exploration capabilities of each node. GTSR-RO-1 and GTSR-MP-4 extend the perception range of agents to extract more information for routing, thus successfully achieving delays below 100 ms on several unseen topologies. However, they still encounter failures or fluctuations on certain $\mathcal{G}_d$. In comparison, GTSR incorporates long-range perception in both the global message-passing and readout modules and establishes a MADRL cooperation mechanism to facilitate the learning of general perception policy. Therefore, GTSR maintains equivalent delays below 50 ms on all 21 unseen $\mathcal{G}_d$, demonstrating its superior robustness. The advantages of GTSR are also reflected in Figure 11f, where the fourth and second-to-last curves imply that GTSR experiences congestion and a rise in latency during the first few steps when encountering certain unknown topologies. Fortunately, equipped with the superior capability of long-range perception, agents in GTSR are able to effectively detect the

direction of congestion and adjust their route policies accordingly, ultimately eliminating the congestion and reducing the delay.

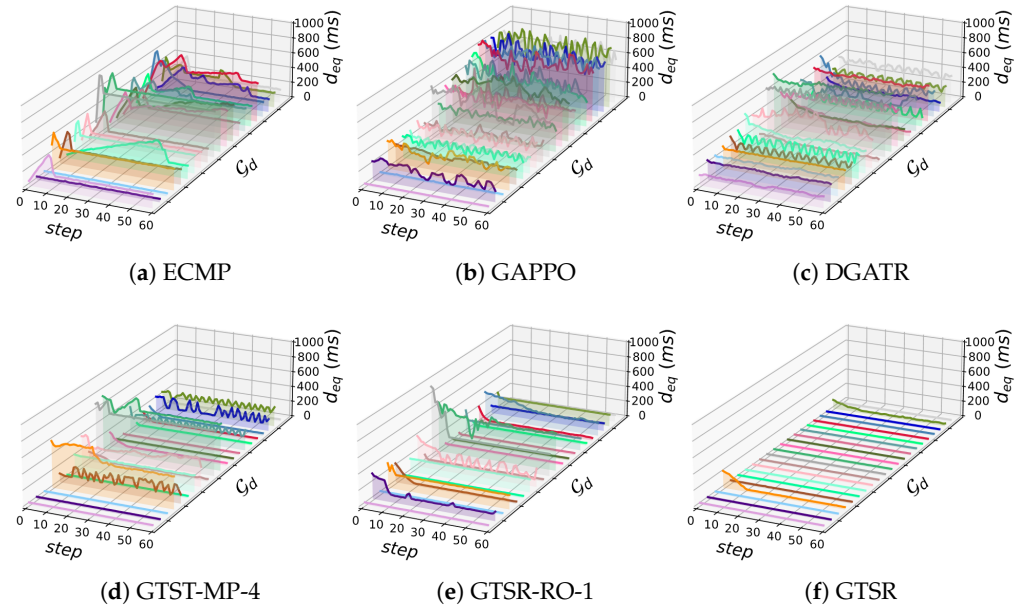


Figure 11. Detailed equivalent delay d_{eq} of each algorithm during testing on 21 unseen topologies $\mathcal{G}_d$ of Nsfnet involving a single link failure. Each line with a distinctive color in the 3D subplot represents d_{eq} over 60 steps on a specific $\mathcal{G}_d$.

Figure 12 and Table 2 present the testing performance of all algorithms on unseen topologies after one or two link failures. In Figure 12, we illustrate the average E2E delay and packet loss rate over 60 steps on each $\mathcal{G}_d$, utilizing bar plots and box plots to showcase the mean and distribution, respectively. Table 2 records the mean, variance, and confidence intervals of testing results for each algorithm in multiple rounds of repeated experiments, where the confidence intervals are mainly represented as the probability of latency below 100 ms or packet loss rate below 10%. It is observed that GAPPO performs worst due to the weakness of a single agent. DGATR presents improvements through MADRL and achieves better packet loss rates than ECMP. GTSR-MP-4 and GTSR-RO-1 further decrease both E2E delay and packet loss rate by expanding the perception range. Compared to all baselines, GTSR reduces E2E delay by at least 47% and packet loss rate by at least 10% on unseen topologies, demonstrating its superior robustness. We also notice a positive correlation between expanding the perception range and improving robustness, since a larger perception range demands a stronger capability for comprehensive information processing and forces agents to learn general routing strategies.

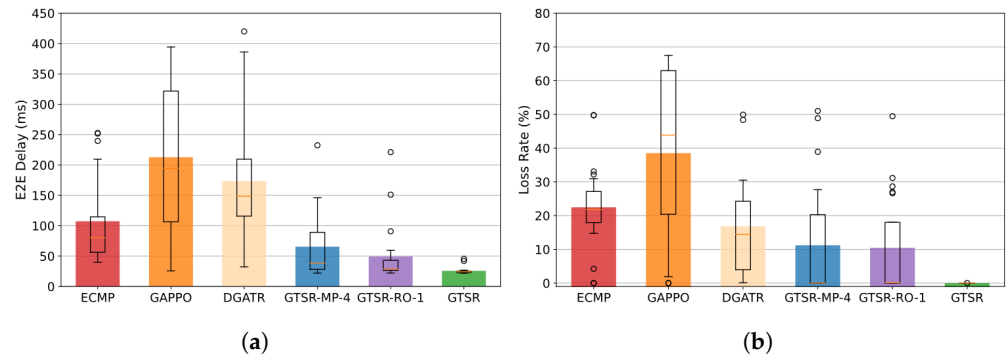


Figure 12. Overview of testing results of all algorithms on all unseen topologies of Nsfnet involving one or two link failures: (a) E2E delay. (b) Packet loss rate.

Table 2. Summary of statistical results of all algorithms during testing on unseen topologies of Nsfnet.

Algorithm	E2E Delay			Packet Loss Rate		
	Mean (ms)	Variance	<100 ms	Mean (ms)	Variance	<10%
ECMP	107.35	1.47×10^4	67.46%	22.46	2.30×10^2	23.25%
GAPPO	212.87	1.86×10^4	24.76%	38.51	5.78×10^2	18.17%
DGATR	173.48	1.25×10^4	30.07%	16.83	2.11×10^2	38.33%
GTSR-MP-4	65.52	6.91×10^3	80.47%	11.20	2.82×10^2	63.33%
GTSR-RO-1	49.42	5.40×10^3	89.41%	10.45	2.08×10^2	59.25%
GTSR	25.74	1.71×10^2	99.20%	0.003	2.47×10^{-4}	99.99%

5.5. Evaluation in Other Real-World Networks

After evaluation on Nsfnet, we extend our experiments to more real-world networks. As shown in Figure 6, four networks of varying sizes are collected from the Topology Zoo dataset, including national backbone networks and intercontinental networks. Table 3 lists the sending rate of uniform traffic, the number of unseen topologies, and the statistical differences between $\mathcal{G}_o$ and $\mathcal{G}_d$. *Betweenness* reflects the average importance or burden for data transmission of all nodes in the graph, which is represented by the maximum and minimum among all $\mathcal{G}_d$, along with the percentage of change compared to $\mathcal{G}_o$. For each network, all algorithms are trained for 36,000 steps on its original topology $\mathcal{G}_o$ and tested for 60 steps on each of its faulty topologies $\mathcal{G}_d$ without retraining.

Table 3. Information for the four real-world networks.

Topology	$\mathcal{G}_o$		$\mathcal{G}_{d1}$		$\mathcal{G}_{d2}$		Sending Rate (Mbps)
	(N, E)	Betweenness	Number	Betweenness	Number	Betweenness	
Sprint	(11, 18)	0.0989	17	[0.1191, 0.0888] [+20.40%, −10.20%]	132	[0.1393, 0.0916] [+40.81%, −7.397%]	0.063
Navigata	(13, 17)	0.1188	11	[0.1282, 0.1200] [+7.843%, +0.9803%]	52	[0.1515, 0.1191] [+27.45%, +0.2614%]	0.029
BtAsiaPac	(20, 31)	0.0733	25	[0.0804, 0.0729] [+9.561%, −0.6327%]	294	[0.0944, 0.0727] [+28.68%, −0.9257%]	0.021
Hurricane-Electric	(24, 37)	0.0800	30	[0.0871, 0.0790] [+8.847%, −1.234%]	427	[0.0989, 0.0768] [+23.66%, −3.997%]	0.014

Figure 13 presents the testing results of all algorithms in the four real-world networks. The cumulative distribution function (CDF) is employed to depict the distributions of E2E delay and packet loss rate across all unseen $\mathcal{G}_d$ of each network. A curve closer to the y-axis indicates more samples with low delay or packet loss rate. ECMP performs poorly, especially in terms of packet loss rate, due to the congestion on the shortest paths. This issue is particularly severe in Navigata, where there are fewer paths to be selected. The single-agent GAPPO suffers from the lack of independent decision-making capabilities, leading to failures on many unseen $\mathcal{G}_d$. MADRL-based baselines generally perform better in smaller-scale networks, since smaller graphs demand less long-range perception. In Sprint and Navigata, DGATR, GTSR-MP-4, and GTSR-RO-1 achieve delays below 100 ms on all unseen faulty topologies. However, they still encounter high packet loss rates on several $\mathcal{G}_d$. In contrast, GTSR stands out as the only algorithm capable of maintaining the lowest E2E delay below 100 ms and the lowest packet loss rate below 0.02% across all unseen $\mathcal{G}_d$ in every network. These results reveal the superior performance and robustness of GTSR on real-world network topologies. When applied to real-world environments involving more variability, baselines are more likely to exhibit worse performance, while GTSR still

holds the potential to adapt to complex environments now that it possesses the capabilities of perceiving traffic and learning the general routing policy.

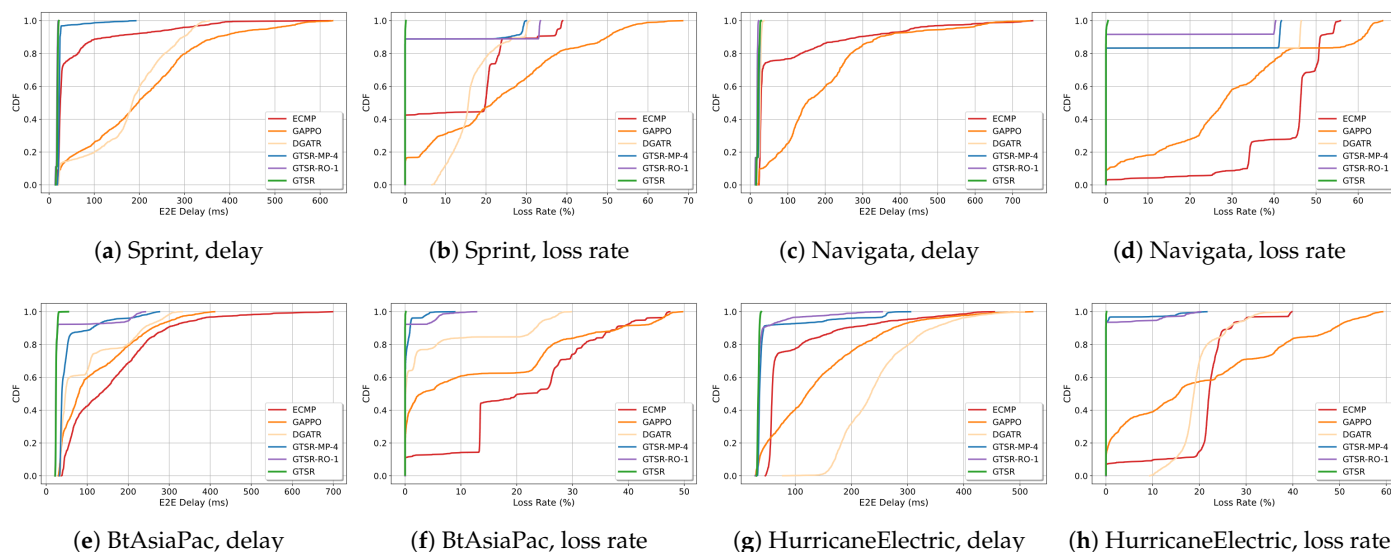


Figure 13. Testing results of all algorithms on unseen topologies from four real-world networks in the form of CDF, where each line presents the cumulative distribution for an algorithm during testing on all unseen topologies from a network.

6. Discussion

After evaluating the proposed routing algorithms with baselines, we identify several key issues as follows:

Advantages of GTSR: The core of GTSR is to learn the nature of packet routing, i.e., the mapping from surrounding traffic to forwarding decisions. Therefore, we carefully design the modules in GTSR, dedicated to equipping agents with the capability of perceiving more information and extracting crucial features for routing. Benefiting from these designs, GTSR is able to effectively detect congestion from long-range perception and reduce the corresponding link weights, thereby alleviating congestion and reducing latency and packet loss rate. Furthermore, this learned routing policy is universal and irrelevant to a particular topology, and thus GTSR is capable of adapting to unseen topology changes.

Limitations of GTSR: Given that GTSR expands the perception range and the amount of information, the first limitation lies in the increased computational complexity, especially in the readout module. The second limitation is the reliance on a central controller. Agents are established within the controller, and all routers also need to communicate with the controller for every step of decision-making, leading to a large communication overhead.

Applications and challenges for real-world networks: Equipped with the capability of learning universal routing strategies and superior robustness, GTSR has a promising prospect in real-world environments, such as 5G, IoT, and other practical applications involving possible connectivity changes. The challenge is to fill the gap between the simulated and real-world environments, requiring extra designs for DRL formulation and fine-tuning for specific demands.

Future research directions: Aiming at the above limitations and challenges, future research will focus on the following: (1) Exploring distributed multi-agent architectures to break the reliance on a central controller, enabling routers to make decisions only through local communication. (2) Evaluation in environments with more complex changes and traffic, optimizing the designs of the GNN and MADRL. (3) Improving MADRL cooperation

mechanisms, highlighting different roles of agents, as well as commonalities and differences among nodes.

7. Conclusions

In this paper, we propose a novel algorithm named GTSR to enhance the robustness of routing against topology changes in SDNs. GTSR emphasizes the perception of traffic at each node for packet routing in a multi-agent architecture. Specifically, we leverage the Graph Transformer to establish a global message-passing mechanism with a virtual star node, enabling long-range perception. In addition, a path-based readout method is proposed to promote the detection of potential congestion along paths. Furthermore, a federated learning-based MADRL cooperation mechanism is designed to facilitate the learning of universal perception strategies while retaining independent decision-making. We construct an SDN simulation environment and conduct extensive experiments on multiple real-world network topologies. It is demonstrated that GTSR trained on a single topology is capable of adapting to unseen topology changes without retraining, showcasing its superior robustness compared to all baselines, as well as the promising prospect for practical applications. Future work will focus on addressing the existing limitations of GTSR, such as exploring fully distributed architectures to release the reliance on a central controller, improving the designs of the GNN and MADRL for scenarios involving complex changes and traffic, and utilizing the commonalities and differences among nodes to enhance MADRL cooperation for learning routing policies.

Author Contributions: Conceptualization, X.L. and J.L. (Jun Liu); Data curation, X.L. and J.L. (Junze Li); Formal analysis, X.L.; Funding acquisition, J.L. (Jun Liu); Investigation, X.L.; Methodology, X.L. and J.L. (Junze Li); Project administration, J.L. (Jun Liu); Resources, J.Z.; Software, J.Z.; Supervision, J.L. (Jun Liu); Validation, J.L. (Junze Li) and J.Z.; Visualization, X.L.; Writing—original draft, X.L.; Writing—review and editing, X.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was partially funded by the National Natural Science Foundation of China under the Grant No. 62371057 and the BUPT Excellent Ph.D. Students Foundation No. CX2023215.

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

CDF	Cumulative Distribution Function
DRL	Deep Reinforcement Learning
E2E	End-to-End
FC	Fully Connected
GNN	Graph Neural Network
MADRL	Multi-Agent Deep Reinforcement Learning
MPNN	Message Passing Neural Network
NN	Neural Network
QoS	Quality of Service
SADRL	Single-Agent Deep Reinforcement Learning
SDN	Software Defined Network
TM	Traffic Matrix

References

1. Moy, J. OSPF Version 2. Technical Report, 1997. Available online: <https://www.rfc-editor.org/rfc/rfc2178.html> (accessed on 20 January 2025).
2. Hopps, C. Analysis of an Equal-Cost Multi-Path Algorithm. Technical Report, 2000. Available online: <https://www.rfc-editor.org/rfc/rfc2992.html> (accessed on 20 January 2025).
3. Kreutz, D.; Ramos, F.M.; Verissimo, P.E.; Rothenberg, C.E.; Azodolmolky, S.; Uhlig, S. Software-defined networking: A comprehensive survey. *Proc. IEEE* **2014**, *103*, 14–76. [\[CrossRef\]](#)
4. Arulkumaran, K.; Deisenroth, M.P.; Brundage, M.; Bharath, A.A. Deep reinforcement learning: A brief survey. *IEEE Signal Process. Mag.* **2017**, *34*, 26–38. [\[CrossRef\]](#)
5. Xiao, Y.; Liu, J.; Wu, J.; Ansari, N. Leveraging deep reinforcement learning for traffic engineering: A survey. *IEEE Commun. Surv. Tutorials* **2021**, *23*, 2064–2097. [\[CrossRef\]](#)
6. Guo, Y.; Lin, B.; Tang, Q.; Ma, Y.; Luo, H.; Tian, H.; Chen, K. Distributed Traffic Engineering in Hybrid Software Defined Networks: A Multi-agent Reinforcement Learning Framework. *IEEE Trans. Netw. Serv. Manag.* **2024**, *21*, 6759–6769. [\[CrossRef\]](#)
7. You, X.; Li, X.; Xu, Y.; Feng, H.; Zhao, J.; Yan, H. Toward packet routing with fully distributed multiagent deep reinforcement learning. *IEEE Trans. Syst. Man Cybern. Syst.* **2020**, *52*, 855–868. [\[CrossRef\]](#)
8. Rusek, K.; Suárez-Varela, J.; Mestres, A.; Barlet-Ros, P.; Cabellos-Aparicio, A. Unveiling the potential of graph neural networks for network modeling and optimization in SDN. In Proceedings of the 2019 ACM Symposium on SDN Research, San Jose, CA, USA, 3–4 April 2019; pp. 140–151.
9. Rusek, K.; Suárez-Varela, J.; Almasan, P.; Barlet-Ros, P.; Cabellos-Aparicio, A. RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN. *IEEE J. Sel. Areas Commun.* **2020**, *38*, 2260–2270. [\[CrossRef\]](#)
10. Scarselli, F.; Gori, M.; Tsoi, A.C.; Hagenbuchner, M.; Monfardini, G. The graph neural network model. *IEEE Trans. Neural Netw.* **2008**, *20*, 61–80. [\[CrossRef\]](#)
11. Cui, Z.; Henrickson, K.; Ke, R.; Wang, Y. Traffic graph convolutional recurrent neural network: A deep learning framework for network-scale traffic learning and forecasting. *IEEE Trans. Intell. Transp. Syst.* **2019**, *21*, 4883–4894. [\[CrossRef\]](#)
12. Hope, O.; Yoneki, E. GDDR: GNN-based data-driven routing. In Proceedings of the 2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS), Washington, DC, USA, 7–10 July 2021; pp. 517–527.
13. Swaminathan, A.; Chaba, M.; Sharma, D.K.; Ghosh, U. GraphNET: Graph neural networks for routing optimization in software defined networks. *Comput. Commun.* **2021**, *178*, 169–182. [\[CrossRef\]](#)
14. Mai, X.; Fu, Q.; Chen, Y. Packet routing with graph attention multi-agent reinforcement learning. In Proceedings of the 2021 IEEE Global Communications Conference (GLOBECOM), Madrid, Spain, 7–11 December 2021; pp. 1–6.
15. Almasan, P.; Suárez-Varela, J.; Rusek, K.; Barlet-Ros, P.; Cabellos-Aparicio, A. Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case. *Comput. Commun.* **2022**, *196*, 184–194. [\[CrossRef\]](#)
16. He, Q.; Wang, Y.; Wang, X.; Xu, W.; Li, F.; Yang, K.; Ma, L. Routing optimization with deep reinforcement learning in knowledge defined networking. *IEEE Trans. Mob. Comput.* **2023**, *23*, 1444–1455. [\[CrossRef\]](#)
17. Chen, B.; Zhu, D.; Wang, Y.; Zhang, P. An approach to combine the power of deep reinforcement learning with a graph neural network for routing optimization. *Electronics* **2022**, *11*, 368. [\[CrossRef\]](#)
18. Li, X.; Xiao, Y.; Liu, S.; Lu, X.; Liu, F.; Zhou, W.; Liu, J. GAPPO-A Graph Attention Reinforcement Learning based Robust Routing Algorithm. In Proceedings of the 2023 IEEE 34th Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC), Toronto, ON, Canada, 5–8 September 2023; pp. 1–7.
19. Bhavanasi, S.S.; Pappone, L.; Esposito, F. Dealing with changes: Resilient routing via graph neural networks and multi-agent deep reinforcement learning. *IEEE Trans. Netw. Serv. Manag.* **2023**, *20*, 2283–2294. [\[CrossRef\]](#)
20. Bernárdez, G.; Suárez-Varela, J.; López, A.; Shi, X.; Xiao, S.; Cheng, X.; Barlet-Ros, P.; Cabellos-Aparicio, A. Magnneto: A graph neural network-based multi-agent system for traffic engineering. *IEEE Trans. Cogn. Commun. Netw.* **2023**, *9*, 494–506. [\[CrossRef\]](#)
21. Gilmer, J.; Schoenholz, S.S.; Riley, P.F.; Vinyals, O.; Dahl, G.E. Neural message passing for quantum chemistry. In Proceedings of the International Conference on Machine Learning, Sydney, Australia, 6–11 August 2017; pp. 1263–1272.
22. Huang, W.; Rong, Y.; Xu, T.; Sun, F.; Huang, J. Tackling over-smoothing for general graph convolutional networks. *arXiv* **2020**, arXiv:2008.09864.
23. Wu, Q.; Zhao, W.; Yang, C.; Zhang, H.; Nie, F.; Jiang, H.; Bian, Y.; Yan, J. Simplifying and empowering transformers for large-graph representations. *Adv. Neural Inf. Process. Syst.* **2024**, *36*.
24. Rampásek, L.; Galkin, M.; Dwivedi, V.P.; Luu, A.T.; Wolf, G.; Beaini, D. Recipe for a general, powerful, scalable graph transformer. *arXiv* **2022**, arXiv:2205.12454.
25. Varga, A.; Hornig, R. An overview of the OMNeT++ simulation environment. In Proceedings of the 1st International ICST Conference on Simulation Tools and Techniques for Communications, Networks and Systems, Marseille, France, 3–7 March 2010.
26. Stampa, G.; Arias, M.; Sánchez-Charles, D.; Muntés-Mulero, V.; Cabellos, A. A deep-reinforcement learning approach for software-defined networking routing optimization. *arXiv* **2017**, arXiv:1709.07080.

27. Valadarsky, A.; Schapira, M.; Shahaf, D.; Tamar, A. Learning to route. In Proceedings of the 16th ACM Workshop on Hot Topics in Networks, Palo Alto, CA, USA, 30 November–1 December 2017; pp. 185–191.
28. Zhang, W.; Yang, D.; Wu, W.; Peng, H.; Zhang, N.; Zhang, H.; Shen, X. Optimizing federated learning in distributed industrial IoT: A multi-agent approach. *IEEE J. Sel. Areas Commun.* **2021**, *39*, 3688–3703. [[CrossRef](#)]
29. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal policy optimization algorithms. *arXiv* **2017**, arXiv:1707.06347.
30. Knight, S.; Nguyen, H.X.; Falkner, N.; Bowden, R.; Roughan, M. The internet topology zoo. *IEEE J. Sel. Areas Commun.* **2011**, *29*, 1765–1775. [[CrossRef](#)]
31. Fey, M.; Lenssen, J.E. Fast graph representation learning with PyTorch Geometric. *arXiv* **2019**, arXiv:1903.02428.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.